



Evaluation: Test Sets

Prof. Gilad Gressel
Amrita School of Engineering

Introduction

- We discuss the need for test sets
- How do we create test sets
- When is it appropriate to use them

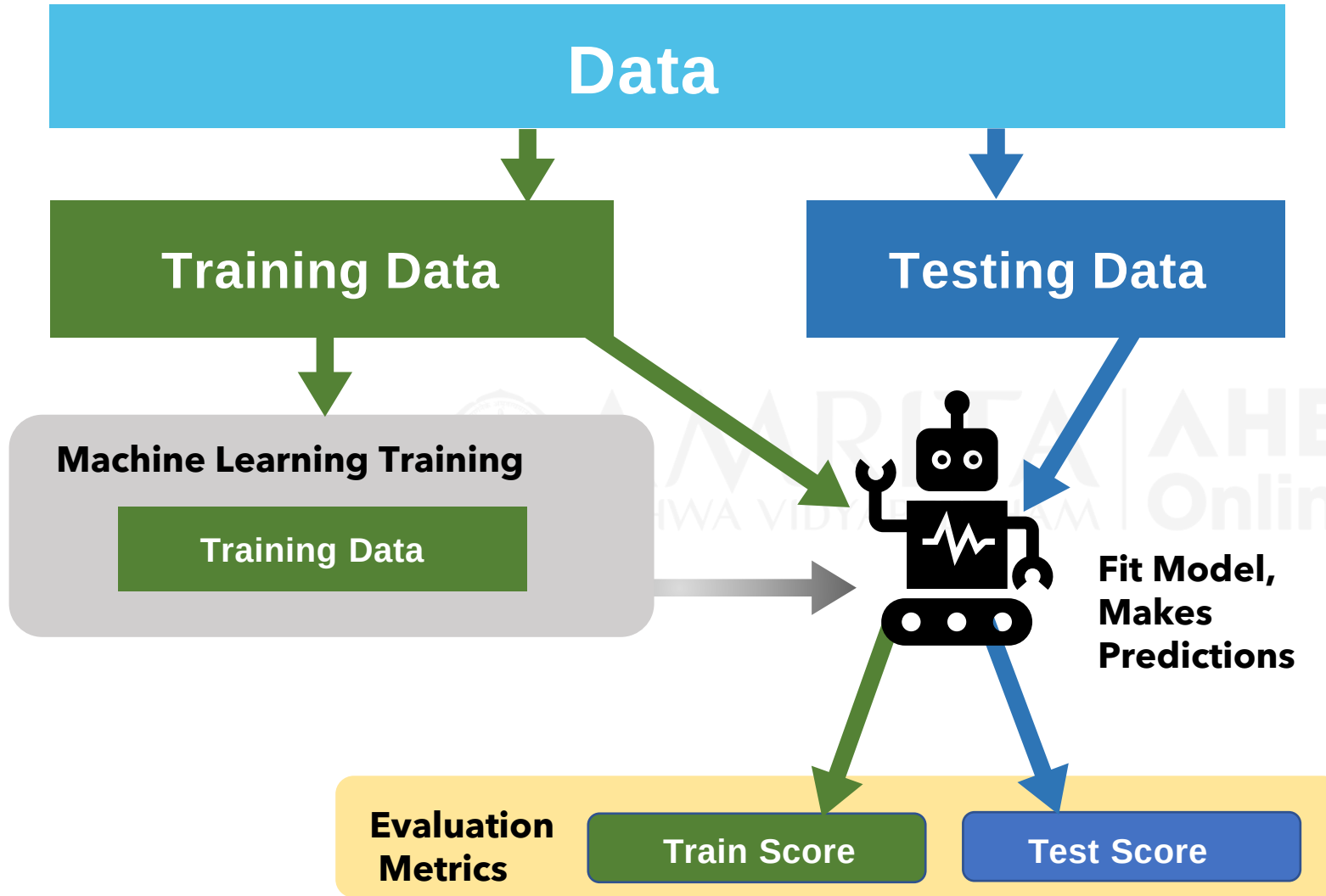
Place your
Webcam Video here
Size 100%

Test Sets



- We need to evaluate how our model will perform *tomorrow*
 - But we don't have access to tomorrows data
- Do we have access to data that represents our products target?
 - If we want to make a model that classifies whether or not a picture is a "cat" or "dog", do we have access to the photos people will use?
 - Malware detection - do we have access to the malware people will run into?
- Testing sets are our "best attempt" at **simulating** tomorrows input data.

Train Test Split



- Split your data into two parts
- **Hide** away the testing set
- Train on the training set - Get a model
- Make predictions on **both** sets
- Evaluate on **both** sets of predictions.
- This is the primary way to **detect** overfitting

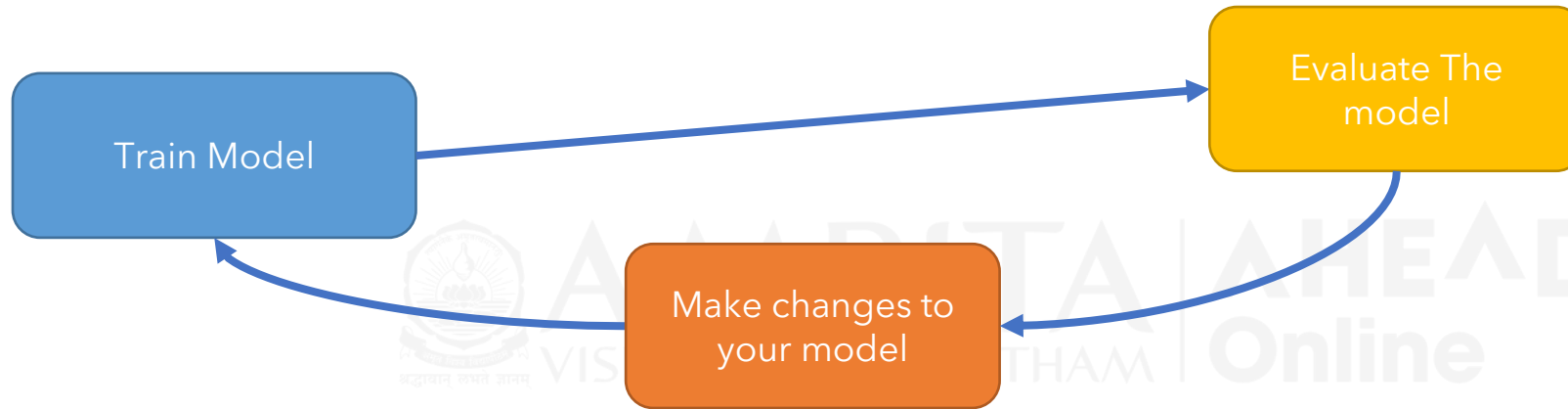


"Only use a test set Once"

-Every Machine Learning Teacher

How often can we evaluate?

- Ideally you only evaluate *once*, because



- The looping pattern of evaluating your model, then making changes, leads to overfitting to the test set
- Every time you evaluate against the test set – it becomes less effective as a simulation about tomorrow's data
- The more we *know* about the test set, the effective it is at helping us evaluate our models

Conclusion

- We formalized testing sets
- Use your test set sparingly
- Be careful not to overfit to your test set

Place your
Webcam Video here
Size 100%



Train Test Split Options

Prof. Gilad Gressel
Amrita School of Engineering

Introduction

- train_test_split
- Test set size
- Shuffling
- Stratification
- Random States

Place your
Webcam Video here
Size 100%

train_test_split



- Scikit-Learn has an excellent module that splits your data into two sets
- train_test_split, will take as input:
 - data
 - labels
- train_test_split will return:
 - training_data
 - training_labels
 - testing_data
 - testing_labels



Test Set Size



- Depends on situation
 - You need enough data to train reliably
 - But you also need enough to test!
 - Generally more is needed for training
- 20-30% for testing is common

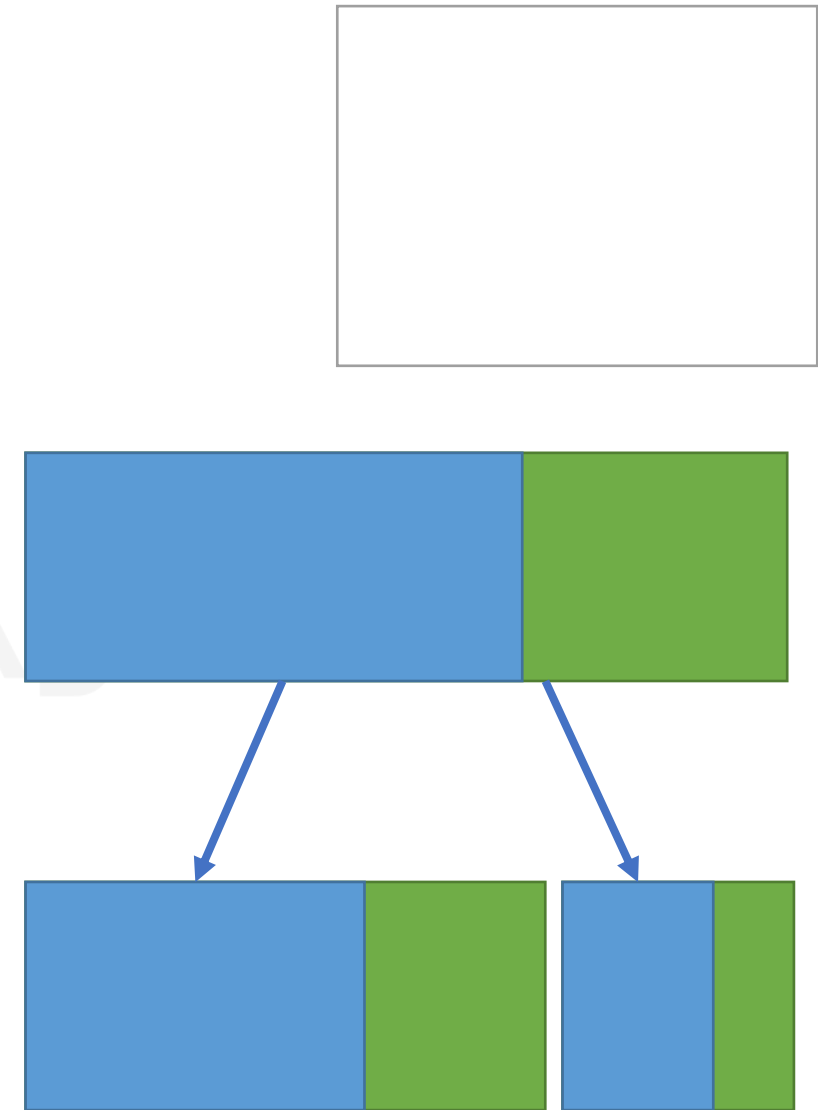
Shuffling Your Data



- Randomizing your data during split
- It's a good idea because if your data is organized in any kind of fashion – we probably don't want that order preserved.
 - aaaaaaabbabbbbbb → we want a mix of both cases in train / test
- Edge case:
 - Time-series data should **never** be shuffled
 - You don't want to mix the future and the past up!

Stratification

- For classification tasks, stratification makes sure that you draw evenly from all the classes
- If you have 30-70 ratio of spam-ham in your data, then that ratio will get maintained in your train-test-split
- If you want to model some random factors, then perhaps you would leave this off.



Random State



- Setting this makes things **not** random (deterministic), you will get the *exact* same split every time you run it
- GOOD if:
 - You need to eliminate variations to debug something
 - You are teaching a tutorial and you want everyone to get the same results.
 - You plan on reusing your test set again later on and want to make sure you can get the exact same one again. But!.. we shouldn't reuse test sets
- Bad if:
 - You are trying to find your models overall performance

Tips on Train/test Splits



- Try to draw from the same distribution as the training set.
 - If it's malware detector and you **do have** some recent live data, then **use it** in combination with downloaded older data.
 - Include the recent live data, evenly both in training and testing.
- Make sure that **no information** from your training set **ever** gets into your testing set.
 - Feature Scaling
 - Feature Extraction
 - Time series
 - Use scikit learn pipelines to avoid this problem (more on this later)

Conclusion

- Shuffling
- Stratification
- Random States
- Some tips!

Place your
Webcam Video here
Size 100%



Baseline Models

Prof. Gilad Gressel
Amrita School of Engineering

Introduction

- How can we be sure our model has learned anything?
- Baseline models help us know if we are good or not

Place your
Webcam Video here
Size 100%

Is it good?



- How do we know if our models performance is any good?
- How do we know if our model **learned** anything?
- Good is a relative word, so we need to **compare** it with something

Things to Compare Against



- Existing system for evaluation
- State of the art in your problem area (academic)
- Make a baseline model
 - Random Guessing
 - Majority Rule
 - Any rule-based idea you have

Baseline Model



- If you can beat your baseline, then you have something that is **good**
- Or at least...better
- Scikit learn implements the DummyClassifier and DummyRegressor for exactly this purpose

Conclusion

- We discovered baseline models
- By comparing against simple rule based strategies we can tell if our fancy ML model is any good

Place your
Webcam Video here
Size 100%